TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

PROJECT SYNOPSIS

Student Details

Institute/University: Pimpri Chinchwad College of Engineering (PCCOE), Pune

Programme / Batch: B.Tech. Computer Engineering(Regional Language), Academic Year 2026-27

Student Name: Tanishka Kadam

PRN: 124B2D001

1. Case Study ID and Project Title / Topic

Case Study ID (CS1 to CS5): CS1

Project Title / Topic: AutoDoc AI: AUTOSAR HLD Document Analysis Assistant

2. Objectives

Objectives

- To develop a web application for uploading automotive HLD documents in PDF format and asking questions about them in plain English.
- To extract text from the document while keeping track of the page on which each piece of text appears.
- To implement semantic search using sentence embeddings and a vector database.
- To generate answers using only the retrieved passages and to state clearly when the document does not contain the requested information.
- To display the source passages and page references along with every answer.
- To test the system with predefined questions and expected answers and to record its limitations.

3. Scope

In Scope

- Uploading an automotive HLD document in PDF format through a Streamlit interface.
- Extracting the text of each page with PyMuPDF, storing the page number, and dividing the text into smaller passages.
- Semantic retrieval of relevant passages using sentence embeddings and ChromaDB.
- Answer generation with the Gemini API from the retrieved passages, shown together with the source passages and page references.
- Testing with predefined questions and expected answers, including questions whose answers are not present in the document.

Out of Scope

- OCR for scanned pages.

- Extraction of tables and diagrams.
- Automatic listing of components and interfaces, and comparison of document revisions.

4. Proposed Approach

Pre-trained model

Google Gemini model: "gemini-3.8-flash". The model is accessed through the Gemini API and hosted by Google; it is not run locally.

Embedding model

Sentence Transformers model all-MiniLM-L6-v2. It converts each passage of the document and each user question into a numerical vector.

Vector store

ChromaDB. It stores the vectors along with their page information and returns the passages most similar to the question.

RAG pipeline and/or AI-agent design

The system is based on Retrieval-Augmented Generation (RAG). The text of each page is extracted, divided into passages, converted into vectors and stored in ChromaDB. A question is converted into a vector in the same way, and the most similar passages are retrieved.

The retrieved passages are sent to the Gemini API with an instruction to answer only from the given text and to say so if the answer is not available. The application then shows the answer, the passages used and their page numbers.

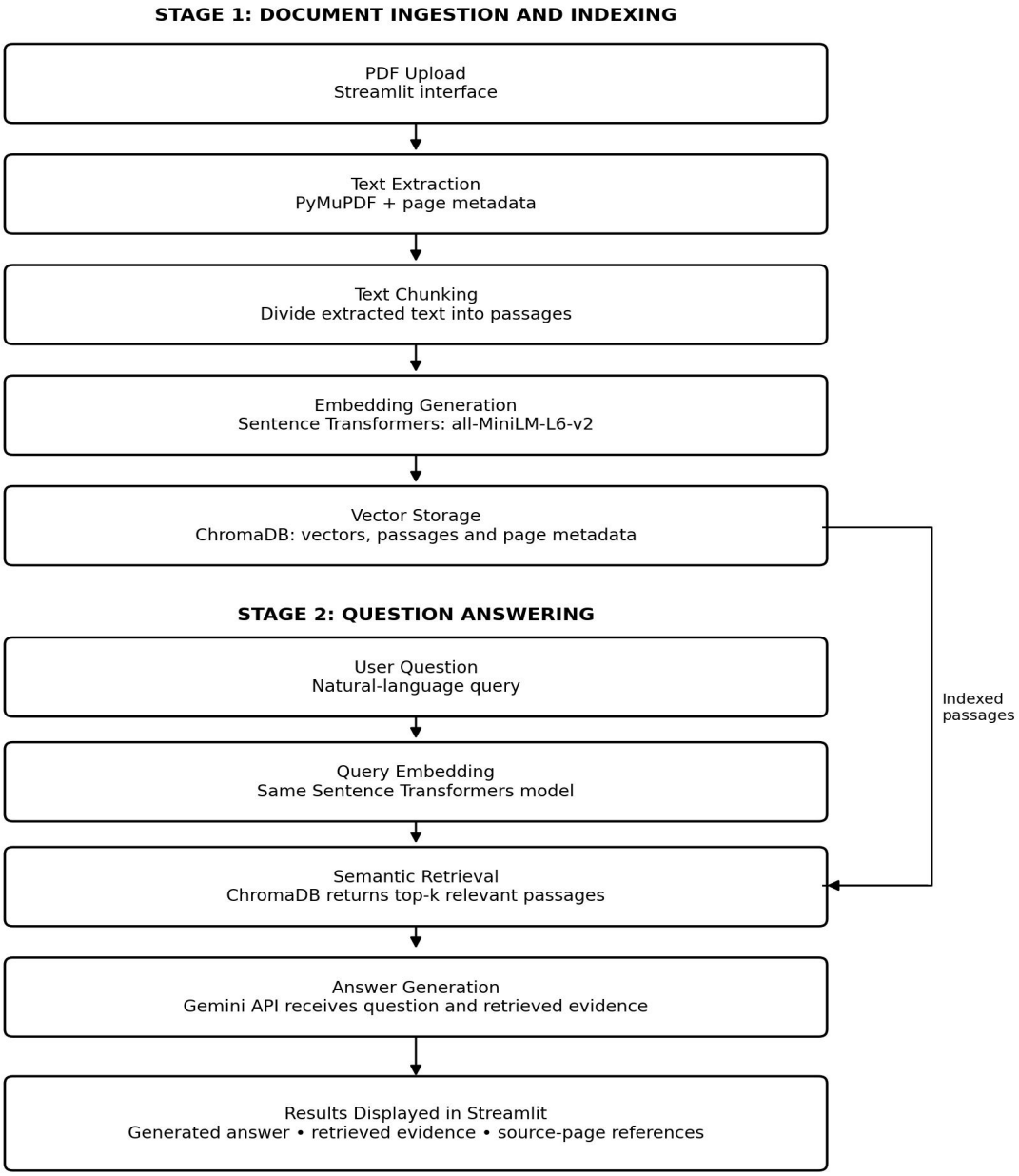
Service layer and UI

Python with Streamlit for the user interface. PDF text is extracted with PyMuPDF and the code is developed in Visual Studio Code. python-dotenv is also used.

Tools / deterministic rules

No separate tools are used. The only fixed rule is the instruction given to the language model to answer only from the retrieved text and to say so when the answer is not available.

Proposed System / Architecture



5. Input Data / Knowledge Base

Data / Knowledge Sources

A synthetic HLD document describing an obstacle detection and automatic braking support system, prepared for demonstration. It is not an official AUTOSAR specification and does not represent a validated safety system.

Approximate Size and Format

The demonstration dataset consists of one synthetic automotive High-Level Design (HLD) document in PDF format, containing 3 pages. The document describes an obstacle detection and automatic braking support system, including component responsibilities, message interactions, and predefined evaluation questions.

The document is synthetic and intended for educational demonstration only; it is not an official AUTOSAR specification or a validated automotive safety document.

Preprocessing and EDA Planned

The text of each page is extracted with PyMuPDF and the page number is stored with it. The text is then divided into smaller passages and each passage is converted into a vector.

6. Expected Outcomes and Evaluation Plan

Expected Outputs

A working web application that answers questions about an uploaded HLD document.

Answers that are based on retrieved passages and come with page references.

Test results showing how the system performs on known questions, including questions that the document cannot answer.

A documented project containing the source code, sample document, test questions and usage instructions.

Evaluation Method

The initial functional evaluation uses predefined questions and manually prepared expected answers from the synthetic HLD document.

1. Answer correctness: Number of test questions whose observed answers match the expected answers.
2. Retrieval relevance: Whether the retrieved passages contain information relevant to the question and support the expected answer.
3. Citation correctness: Whether the displayed page references point to pages that support the generated answer.
4. Handling of unavailable information: Whether the system clearly states that the document does not provide the requested information instead of inventing a value.
5. Reliability: Record API errors, failed requests, and successful retries separately from answer correctness.

Five initial functional tests produced the expected answers during observed runs. This is a preliminary result from a small test set, not a general accuracy estimate. Citation correctness must be independently checked against the source PDF. Further evaluation on a larger set of questions and authorized documents is required.

Responsible AI Measures

Grounding and citations: answers are generated only from the retrieved passages and are shown with the source passages and page numbers. The page references are to be checked against the original PDF.

Privacy: documents are sent to an external service (Gemini API), so only non-confidential or synthetic documents are used.

Human oversight: the answers are meant to assist engineers and must be reviewed by a qualified person before being used for any design decision.

MLOps / Deployment Elements Planned

The project is organized with separate source code, input data, configuration guidance, evaluation results, and documentation to support reproducibility. The dependency list is maintained in requirements.txt, and API credentials are configured through environment variables rather than embedded in source code. Version control can be maintained using Git. The current prototype runs locally through Streamlit and uses the externally hosted Gemini API for answer generation. Docker-based deployment and automated experiment tracking are not implemented in the current version. Future improvements may include dependency version pinning, automated testing, containerization, and systematic evaluation tracking.

7. Planned AI-Tool Usage

Mention the AI tools / models used for coding assistance and specify the purpose and project artifact affected. This information will be carried into the final AI-tool usage declaration.

AI Tool / Model

ChatGPT (OpenAI), Google Gemini API.

Purpose

ChatGPT was used for development guidance, code assistance, debugging support, documentation preparation, and evaluation planning. Google Gemini was used by the application to generate answers from retrieved document passages.

Artifact Affected

Python application code, prompts and configuration, project synopsis, README, evaluation documentation, and technical report, as applicable.

Student Declaration

☑ I will not change the approved case-study scope, major methodology, or implementation approach without prior faculty approval.

☑ I confirm that the work submitted under this synopsis represents my own understanding and contribution, and I will be responsible for explaining every significant part of the selected case study and its implementation.

Name and Signature: Tanishka Kadam



Date: 10/10/2026